Topic: Terraform Plugin Cache — `.terraformrc`, `plugin_cache_dir` & Shared Providers

(Your notes, cleaned up from dictation + screenshots — Lecture 5)

Quick recall

This is the second item left over from Chapter 03's list (provisioners were Chapter 04).

Term	One-liner
Terraform plugin cache	Like a **local storage that saves plugins** to make Terraform faster and more reliable
Provider plugin	The binary Terraform downloads on `terraform init` (e.g. `hashicorp/aws`) that talks to the cloud API
`.terraform/providers`	Where each project keeps its **own copy** of the providers by default
`.terraformrc`	The **Terraform CLI configuration file** in the user's home directory (`~/.terraformrc`). Settings here apply to every project that user runs
`plugin_cache_dir`	The setting in `.terraformrc` that points at the **one central folder** holding the providers
`disable_checkpoint`	The setting in `.terraformrc` that turns off Terraform's "phone home" upgrade/security check

> **Definition (from the lecture's reference):** The Terraform plugin cache is a built-in mechanism used **to store downloaded provider plugins in a centralized local directory** so they can be reused across multiple Terraform projects. By default, every time you run `terraform init`, Terraform downloads a completely separate copy **of each required provider** (AWS, GCP, Azure...) directly into that specific project's `.terraform/providers` directory. Since provider binaries can easily exceed several hundred megabytes, downloading them repeatedly wastes significant **time, network bandwidth, and disk space**.

---

1. The problem: every project downloads the same provider again

Take one example. I have one operating system (one Linux server) where I've created different folders for different client projects: **project P1, project P2, project P3**. Every client has their own Terraform code in their folder, and **every client uses the same cloud provider, AWS**.

Every time we write code for a project and run `terraform init` in it, Terraform downloads all the provider files again. For P1 it downloads the AWS provider. For P2 it fetches the same AWS provider binary again, and the same again for P3. That creates **overhead from duplicate binaries**.

```
One Linux server (cka-w2)
├── P1  (client XYZ, AWS: EC2, S3)  → terraform init → .terraform/  ~800 MB aws provider
├── P2  (client ABC, AWS: VPC, SG)  → terraform init → .terraform/  ~800 MB aws provider (s
└── P3  (client …,   AWS)           → terraform init → .terraform/  ~800 MB aws provider (s
```

Math. Say one plugin file is 1 GB and we have 10 different project folders. If every project pulls that plugin file separately, **10 GB goes to that one plugin**, and it is also downloaded 10 times over the network.

The lab showed this is real. On another box, one project's `.terraform` folder alone was **874 MB**, almost all of it the AWS provider:

```
[root@Devops-server a]# ls -a
.  ..  s3.tf  .terraform  .terraform.lock.hcl
[root@Devops-server a]# du -sh .terraform
874M    .terraform
```

2. The fix: one central folder for plugins

We want a **centralized location where the plugins are kept once**, and every project uses them from there. We do this with a CLI configuration file called `.terraformrc` in the home directory. In it we set `plugin_cache_dir` to the central folder.

Syntax from the Terraform docs ("Configuration File Syntax"). It uses the same **HCL** syntax as `.tf` files but with different attributes and blocks:

```
plugin_cache_dir   = "$HOME/.terraform.d/plugin-cache"
disable_checkpoint = true
```

- `plugin_cache_dir`: the shared cache folder. `$HOME` is expanded, so for root this is `/root/.terraform.d/plugin-cache`.

- `disable_checkpoint`: by default Terraform contacts HashiCorp's *checkpoint* service to check for newer versions and security bulletins. That's the `checkpoint_signature` file already sitting in `~/.terraform.d`. `true` turns that network call off, which helps on locked-down or offline servers.

3. Lab walkthrough (cka-w2, `~/project1`)

Step 1: Set up three project folders and a small config

```
root@cka-w2:~/project1# mkdir a b c
root@cka-w2:~/project1# cd a
root@cka-w2:~/project1/a# vim s3.tf
```

```
resource "aws_s3_bucket" "first" {
  bucket = "linuxterraform12345"
  tags = {
    Name = "my_buck"
  }
}
```

Step 2: `plan` before `init` fails

```
root@cka-w2:~/project1/a# terraform plan
Error: Inconsistent dependency lock file
  - provider registry.terraform.io/hashicorp/aws: required by this configuration but no vers
To make the initial dependency selections that will initialize the dependency lock file, run
  terraform init
```

This isn't about the cache. It's the normal "you haven't run `init` in this folder yet" error: there is no `.terraform.lock.hcl` and no provider selected. A new folder always needs `init` first.

Step 3: Create `~/.terraformrc`

```
root@cka-w2:~# touch .terraformrc
root@cka-w2:~# vim .terraformrc          # paste the two lines from section 2
root@cka-w2:~# cd /root/.terraform.d
root@cka-w2:~/.terraform.d# ls -ltrha
-rw-r--r-- 1 root root  394 Sep 21 17:21 checkpoint_signature
```

At this point `~/.terraform.d` only has `checkpoint_signature`. **There is no `plugin-cache` folder yet.**

Step 4: `terraform init` complains that the cache folder is missing

```
root@cka-w2:~/project1/a# terraform init
There are some problems with the CLI configuration:
Error: The specified plugin cache dir /root/.terraform.d/plugin-cache cannot be opened:
stat /root/.terraform.d/plugin-cache: no such file or directory

As a result of the above problems, Terraform may not behave as intended.

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.66.0...
- Installed hashicorp/aws v6.66.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl ...
Terraform has been successfully initialized!
```

> **Lesson:** Terraform does **not create the cache directory for you**. If it's missing, you get this warning and init falls back to the normal behaviour (download into the project only). Create it once:
>
> ```
> mkdir -p ~/.terraform.d/plugin-cache
> ```

Step 5: With the folder in place, the provider lands in the cache

Once the directory existed and `init` ran again, the provider was stored centrally:

```
root@cka-w2:~/.terraform.d# ls
checkpoint_signature  plugin-cache

root@cka-w2:~/.terraform.d# tree
.
├── checkpoint_signature
└── plugin-cache
    └── registry.terraform.io
        └── hashicorp
            └── aws
                └── 6.66.0
                    └── linux_amd64
                        ├── LICENSE.txt
                        └── terraform-provider-aws_v6.66.0_x5

root@cka-w2:~/.terraform.d# du -sh plugin-cache
822M    plugin-cache
```

The layout inside the cache is always `<registry host>/<namespace>/<provider>/<version>/<os_arch>/`. The same provider at a different version,

or for a different OS/CPU, gets its own folder.

Step 6: The project now has only a symlink

```
root@cka-w2:~/project1/a# ls -la
-rw-r--r-- 1 root root     112 Sep 28 11:50 s3.tf
drwxr-xr-x 3 root root    4096 Sep 28 12:07 .terraform
-rw-r--r-- 1 root root    1452 Sep 28 12:07 .terraform.lock.hcl
-rw-r--r-- 1 root root 4368729 Sep 28 12:15 terraform.log

root@cka-w2:~/project1/a# tree .terraform
.terraform
└── providers
    └── registry.terraform.io
        └── hashicorp
            └── aws
                └── 6.66.0
                    └── linux_amd64 -> /root/.terraform.d/plugin-cache/registry.terraform.io
7 directories, 0 files
```

This is the key proof: `0 files`. The project's `.terraform` has no 800 MB binary any more, only a **symlink** pointing into the central cache. The 822 MB lives in one place.

(`terraform.log` is still being written because `TF_LOG` / `TF_LOG_PATH` from Chapter 03 are set in this shell. `unset TF_LOG TF_LOG_PATH` when you don't need it.)

Step 7: Other projects reuse it

Now if we go to directory **b** (or c) and run `terraform init`, it **won't keep another copy of the plugin in b**. Terraform reads `.terraformrc`, finds the provider in the plugin cache, and links b's `.terraform/providers/...` to the same folder. The init output says something like `Using hashicorp/aws v6.66.0 from the shared cache directory`. Each individual directory no longer has to download the plugin file again and again.

4. Why it matters

Suppose **100 people** (or 100 projects) are working on this server. With the cache, nobody has to download the plugin file separately:

- **Network bandwidth saved:** the provider is downloaded once, not once per project.
- **Storage saved:** one 800 MB copy instead of N copies.
- **Faster `init`:** creating a symlink is instant, compared with downloading hundreds of MB.
- **More reliable:** fewer registry downloads means fewer failures from a slow or flaky network.

So we cache our plugin file in one centralized location, and it's downloaded only once.

5. Things to know beyond the lab (interview / production)

- **The lock file still decides the version.** The cache only changes *where* the binary comes from. `.terraform.lock.hcl` still pins which version is used, and it should be committed to Git.
- **Terraform ≥ 1.4 and the lock file:** Terraform only trusts a cached provider if it matches a checksum **already recorded in that project's** `.terraform.lock.hcl`. A brand-new folder with no lock file (like `b` in the lab) may still download from the registry once so it can record full checksums. In real repos the lock file is committed, so the cache is used. The escape hatch is `plugin_cache_may_break_dependency_lock_file = true` in `.terraformrc`, which isn't recommended.
- **Environment variable alternative:** `export TF_PLUGIN_CACHE_DIR="$HOME/.terraform.d/plugin-cache"` does the same thing without a config file. It's handy in CI pipelines and Docker images.
- **Windows:** the CLI config file is `%APPDATA%\terraform.rc` (no leading dot).
- **Custom file location:** `TF_CLI_CONFIG_FILE=/path/to/file` overrides where Terraform looks for the CLI config.
- **Per user, per machine:** `$HOME` is different for each Linux user. For many users on one server, point everyone at one shared directory with the right permissions. Terraform doesn't guarantee the cache is safe for many `init`s **at the same time**, so be careful with parallel CI jobs.
- **The cache never cleans itself.** Old versions pile up (`aws/6.66.0`, `aws/6.67.0`, ...). Delete old version folders by hand now and then.
- **For teams or air-gapped setups** the bigger version of this idea is a **provider mirror** (`terraform providers mirror <dir>` plus a `provider_installation { filesystem_mirror / network_mirror }` block in `.terraformrc`), so servers never have to reach the public registry.

Command reference (this chapter)

```
# One-time setup (Linux/macOS)
cat > ~/.terraformrc <<'EOF'
plugin_cache_dir   = "$HOME/.terraform.d/plugin-cache"
disable_checkpoint = true
EOF
mkdir -p ~/.terraform.d/plugin-cache     # Terraform will NOT create it for you

# Or, without a config file
export TF_PLUGIN_CACHE_DIR="$HOME/.terraform.d/plugin-cache"

# Use it
terraform init                           # provider goes into the cache, project gets a syml
tree ~/.terraform.d                      # see registry/namespace/provider/version/os_arch
du -sh ~/.terraform.d/plugin-cache       # one copy (~822M for aws 6.66.0)
tree .terraform                          # linux_amd64 -> /root/.terraform.d/plugin-cache/..
du -sh .terraform                        # tiny now (was 874M without the cache)
```